Predictive: neural models

NN for forecast

Neural networks (NN) are very effective in identifying non linear models.

Even with input data deriving from complex, nonlinear processes, NN are able to combine linear and nonlinear models to get whichever accuracy level is requested (in prediction):

- NN can learn patterns in linear time series
- NN can learn patterns in nonlinear time series
- NN can generalize linear and nonlinear patterns

NN are nonparametric, they do not make hypotheses on noise distribution (gaussian or else).

NN identify a model from input data, essentially defining a model of the generating process.

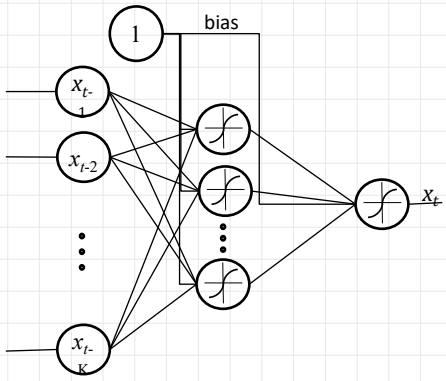
NN for forecast

Given the loosely neurobiological inspiration of the NN model, terminology is peculiar:

NN	Statistic
input nodes	*independent variables*
output nodes	*dependent variables*
weights	*parameters*
training	*identification*

Time-lagged NN

The net is structured according to feedforward scheme, where inputs are associated to past data and the output is the forecast.

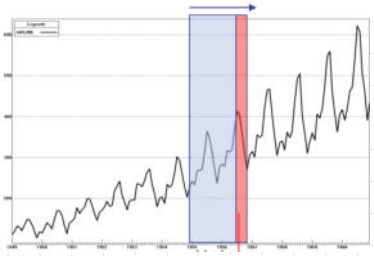


When the input of hidden and output neurons sees only *well-defined subsets* of the output of the neurons of the previous layer and applies them specific operators, the net is named convolutional. These are networks most effective for classification, less so for regression.

Sliding window

Learning:

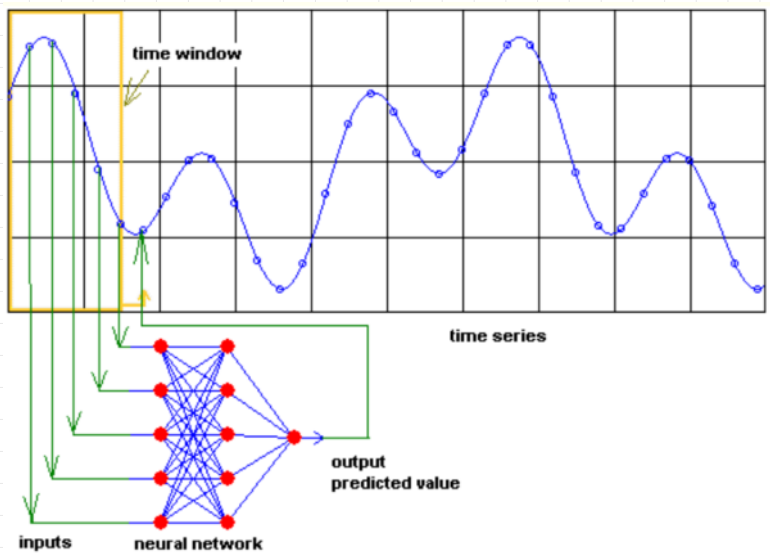
1. A window on past data is shown in input.
2. The corresponding output is computed.
3. The output is compared with its corresponding actual value.
4. *Backpropagation*: weights are changed in order to reduce prediction error.
5. The window is shifted forward, presenting the following input.



N input n output shiftato di 1

Gli n-e ripetuti aiutano nella convergenza

Sliding window



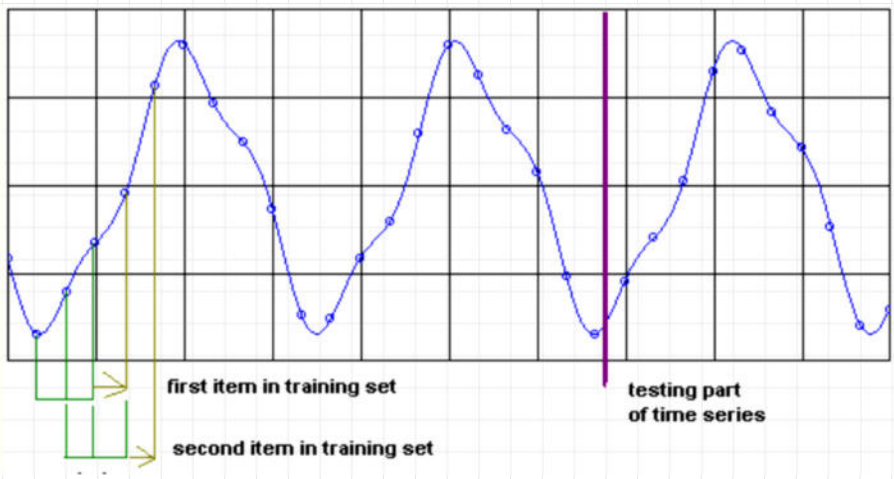
From series to dataset

Transform a data series into a learning dataset where the next datapoint of each record is the unknown.

```
# converts a 1D array of values into two np arrays
# (row vector, if column vector pick first col, add ,0] after look_back])
def create_dataset(arrdata, look_back=1):
        dataX, dataY = [], []
        for i in range(len(arrdata) - look_back):
                a = arrdata[i:(i + look_back)]
                dataX.append(a)
                dataY.append(arrdata[i + look_back])
        return np.array(dataX), np.array(dataY)

# converts a series, 1 col of df values, into a windowed dataframe
def create_dataset2(df, look_back=1,colname='value'):
        cols = ['x{}'.format(x) for x in range(look_back)] + ['y']
        df_win = []
        for i in range(look_back, df.shape[0]):
                df_win.append(df.loc[i-look_back:i, colname].tolist())
        df_win = pd.DataFrame(df_win, columns= cols)
        return df_win
def create_dataset2(df, look_back=1,colname='value'):
3dataset = pd.DataFrame()
   for i in range(lookback, 0, -1):
      dataset['t-' + str(i)] = df.Passengers.shift(i)  # build dataset by columns
   dataset['t'] = df.values
   dataset = dataset[lookback:]  # removes the first lookback columns (no data for
forecast)
```

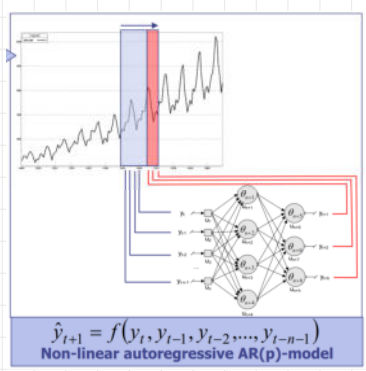
Training and test set



Sliding window

NN are able to approximate any function.

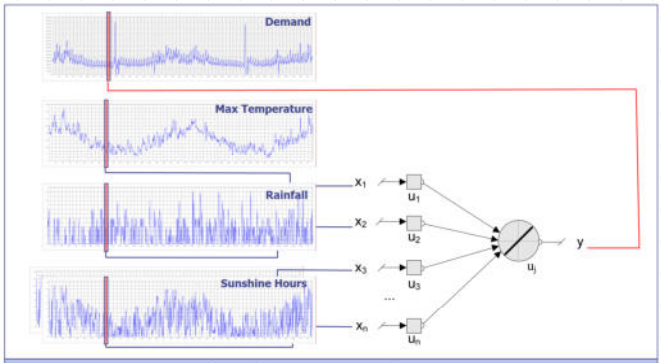
Forecasting can be thought of as the identification of a specific (nonlinear) function.



Only one neuron → AR(p), nonlinear
Feedforward NN (MLP) → combination of AR(p)
Recurrent NN (Elman, Jordan) → ARMA(p,q) nonlinear

NN, linear regression

Example, linear regression including exogenous variables (ARX).



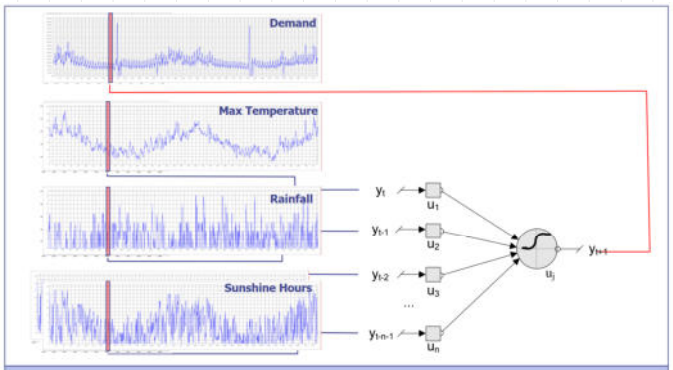
$$\hat{y} = f\left(x_1, x_2, x_3, \ldots, x_n\right)$$

$$\hat{y} = x_1 w_{1j} + x_{2t} w_{2j} + x_{3t} w_{3j} + \ldots + x_n w_{nj} - \theta_j$$

Linear Regression Model

NN, logistic regression

Example, nonlinear regression (logistic) of the same process as slide before.



$$\hat{y}_{t+1} = f\left(y_t, y_{t-1}, y_{t-2}, \ldots, y_{t-n-1}\right)$$

$$\hat{y}_{t+1} = \log\left(\sum_{i=t}^{t-n-1} y_i w_{ij} - \theta_j\right)$$

Nonlinear Multiple (Logistic) Regression Model

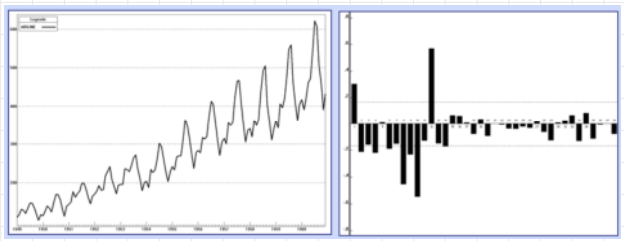
NN, example

Box, Jenkins series, airline passengers

- NN: (12-8-1) 12 input nodes, 8 hidden units, 1 output node
- 12 input values: t, t-1, …, t-11 (last 12 observations)
- Prediction value t+1

Benchmark:

- 132 input values
- 13 time periods (year), monthly data



Python (keras), MLP

```
# Sliding window MLP, Airline Passengers dataset (predicts t+1)
import os, math
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from keras.models import Sequential # pip install k
from keras.layers import Dense      # pip install tenso          dministrator)

# from series of values to windows matrix
def create_dataset(dataset, look_back=1):
        dataX, dataY = [], []
        for i in range(len(dataset) - look_back):
                a = dataset[i:(i + look_back), 0]
                dataX.append(a)
                dataY.append(dataset[i + look_back, 0])
        return np.array(dataX), np.array(dataY)

np.random.seed(550)     # for reproducibility
os.chdir(os.path.dirname(os.path.abspath(__file__)))
df = pd.read_csv('BoxJenkins.csv', usecols=[1], names=['Passengers'], header=0)
dataset = df.values     # time series values
dataset = dataset.astype('float32') # needed for MLP input
```

DO NOT USE KERAS FOR THE FINAL EXAM

Python (Keras), MLP

DO NOT USE KERAS FOR THE FINAL EXAM

```
# split into train and test sets
train_size  = int(len(dataset) -12)
test_size   = len(dataset) - train_size
train, test = dataset[0:train_size,:], dataset[tra[illegible]n(dataset),:]
print("Len train={0}, len test={1}".format(len(train), [illegible]))

# sliding window matrices (look_back = window width); dim = n - [illegible] 1
look_back = 12
testdata = np.concatenate((train[-look_back:],test))
trainX, trainY = create_dataset(train, look_back)
testX, testY   = create_dataset(testdata, look_back)

# Multilayer Perceptron model
loss_function = 'mean_squared_error'
model = Sequential()
model.add(Dense(8, input_dim=look_back, activation='relu')) # 8 hidden neurons
model.add(Dense(1))                                         # 1 output neuron
model.compile(loss=loss_function, optimizer='adam')
model.fit(trainX, trainY, epochs=200, batch_size=2, verbose=2)
```

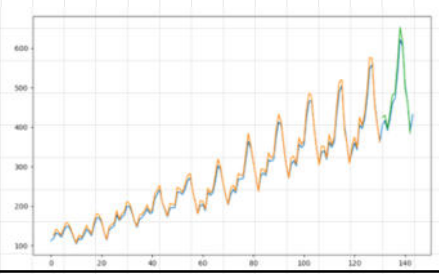

Python (Keras), MLP

DO NOT USE KERAS FOR THE FINAL EXAM

```
# Estimate model performance
trainScore = model.evaluate(trainX, train[illegible]e=0)
print('Train Score: MSE: {0:0.3f} RMSE: ({1:[illegible]rmat(trainScore,
math.sqrt(trainScore)))
testScore = model.evaluate(testX, testY, verbose=0)
print('Test Score: MSE: {0:0.3f} RMSE: ({1:0.3f})'.format(te[illegible]ath.sqrt(testScore)))
# generate predictions for training and forecast for plotting
trainPredict = model.predict(trainX)
testForecast = model.predict(testX)

plt.plot(dataset)
plt.plot(np.concatenate((np.full(look_back-1, np.nan), trainPredict[:,0])))
plt.plot(np.concatenate((np.full(len(train)-1, np.nan), testForecast[:, 0])))
plt.show()
```

This predicts 12 times 1 p[illegible] future, not 12 points in the futu[illegible]



Python (torch), MLP, preliminaries

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.optim as optim
from sklearn.metrics import root_mean_squared_error

# convert an array of values into a dataset matrix
def create_dataset(dataset, look_back=1):
   dataX, dataY = [], []
   for i in range(len(dataset) - look_back):
      a = dataset[i:(i + look_back), 0]
      dataX.append(a)
      dataY.append(dataset[i + look_back, 0])
   return np.array(dataX), np.array(dataY)

if __name__ == '__main__':
   # load the dataset, split into input (X) and output (y) variables
   df = pd.read_csv('BoxJenkins.csv', usecols=[1])
   dataset = df.values.astype('float32') # COLUMN VECTOR !!!
   # split into train and test sets
   train_size  = int(len(dataset) - 12)
   test_size   = len(dataset) - train_size
   train, test = dataset[0:train_size, :], dataset[train_size:len(dataset), :]
   print("Len train={0}, len test={1}".format(len(train), len(test)))
   # reshape into X=t and Y=t+1
   look_back = 12
   testdata  = np.concatenate((train[-look_back:], test))
   trainX, trainy = create_dataset(train, look_back)
```


Python (torch), MLP, the model

```
X = torch.FloatTensor(trainX)
y = torch.FloatTensor(trainy)
# define the model
model = nn.Sequential( nn.Linear(12, 10), nn.ReLU(), nn.Linear(10, 8), nn.ReLU(),
                       nn.Linear(8, 1) # no activation function, allowing for negative outputs
                     )
print(model)
# train the model
loss_fn   = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
print(f"Optimizer type: {type(optimizer)}")
n_epochs = 200
batch_size = 1
for epoch in range(n_epochs):
   for i in range(0, len(X), batch_size):
      Xbatch = X[i:i + batch_size]
      y_pred = model(Xbatch)
      ybatch = y[i:i + batch_size]
      loss = loss_fn(y_pred, ybatch)
      optimizer.zero_grad()           # clear old gradients
      loss.backward()
      optimizer.step()
   print(f'Finished epoch {epoch}, latest loss {loss}')
```

Python (torch), MLP, evaluation

```
# model evaluation as RMSE (no_grad: stop tracking gradient computations)
model.eval() # tells the model we are evaluating, not training
with torch.no_grad():
   y_pred = model(X)  # y_pred is a torch tensor
   rmse = root_mean_squared_error(trainy, y_pred.numpy())
   print(f"Test RMSE: {rmse:.4f}")

# model evaluation as AIC
# Count number of parameters (k)
k = sum(p.numel() for p in model.parameters() if p.requires_grad)
# Get predictions on training data
model.eval()
with torch.no_grad():
   y_pred = model(X)
   rss = torch.sum((y - y_pred) ** 2).item() # both tensors
   n = trainX.shape[0]
   aic = n * np.log(rss / n) + 2 * k
   print(f"AIC: {aic:.2f}")

# generate predictions for training and forecast for plotting
trainPredict = model(X)
predictions  = trainPredict.detach().cpu().numpy()
```


Python (torch), MLP, results

```
# Recursive forecasting for testY. Starting from last window from train (or first from testX)
input_seq = trainX[-1].tolist()
testForecast = []
model.eval()
with torch.no_grad():
   for _ in range(len(test)):
      input_tensor = torch.tensor(input_seq, dtype=torch.float32).unsqueeze(0)  # shape [1,
look_back]
      pred = model(input_tensor).item()  # get scalar prediction
      testForecast.append(pred)

      # Roll the input window forward by 1 step
      input_seq = input_seq[1:] + [pred]
print(testForecast)

# plot train/test data and predictions
plt.plot(dataset,label="actual")
plt.plot(range(12,len(predictions)+12),predictions,label="predict")
plt.plot(range(len(predictions)+12,len(predictions)+24),testForecast,label="forecast")
plt.legend()
plt.show(block=True)
```

Modelling process

Phases for the definition of a neural forecast model :

1) Preprocessing (the same as statistical models)
   - Transforms (diff, log, ...)
   - Scaling, if different variables they must have comparable values
   - Normalization, to [0,1] or [-1,1]
2) Choice of the architecture of the NN
   - Number of neurons, input, hidden, output
   - Number of hidden layers (one: MLP, many: deep network)
   - Processing at the nodes (activation function: lin, logistic, ...)
   - Connections between layers
3) Training
   - Weights initialization (and reinitialization?)
   - Training algorithm (backpropagation, which one?, GLT ...)
   - Parameter setting
4) Use of the NN
5) Validation
   - Choice of the dataset
   - Validation criteria

Data preprocessing

Needed to improve forecast effectiveness. Procedures include:

- Verification, correction, editing (errors in the data, etc.)
- Re-encoding of variables (from categorical to numeric, one-hot)
- Scaling of variables (es. Linear interval scaling,

$$y = y_{min} + (y_{max} - y_{min})\frac{x - x_{min}}{x_{max} - x_{min}})$$

- Normalization ($y = \frac{x-\eta}{\sigma}$)
- Selection of independent variables (PCA, ...)
- Removal of the outliers
- Insertion of missing values (means, regression, default, ...)

Data preprocessing

Data preprocessing of the input vector is a mandatory requirement for the application of MLPs.

As the sigmoid activation functions in the hidden nodes are only defined in the interval of]-1, 1[for the hyperbolic tangent or]0, 1[for the logistic function, input data must be scaled to facilitate learning.

Some authors recommend linear scaling of data into smaller intervals, e.g. [0.2, 0.8], to avoid saturation effects at the asymptotic bounds of the activation functions

Models, heuristic choices

Number of hidden nurons in MLP (with n input nodes):

- 2n+1 (Lippmann 87, Hecht, Nielsen 90, Zhang, Pauwo, Hu 98)
- 2n (Wong 91];
- n (Tang, Fishwick 93);
- n/2 (Kang 91);
- 0.75n (Bailey 90);
- 1.5n - 3n (Kasstra, Boyd 96)

Activation function:

- logistic both in hidden and output nodes (Tang,Fischwick 93, Lattermacher,Fuller 95, Sharda,Patil 92)
- Hyperbolic tangent both in hidden and output nodes (Zhang, Hutchinson 93, DeGroot, Wurtz 91);
- Linear in output nodes (Lapedes, Faber 87, Weigend 91, Wong 90)

To conclude (MLP)

Suggestions:

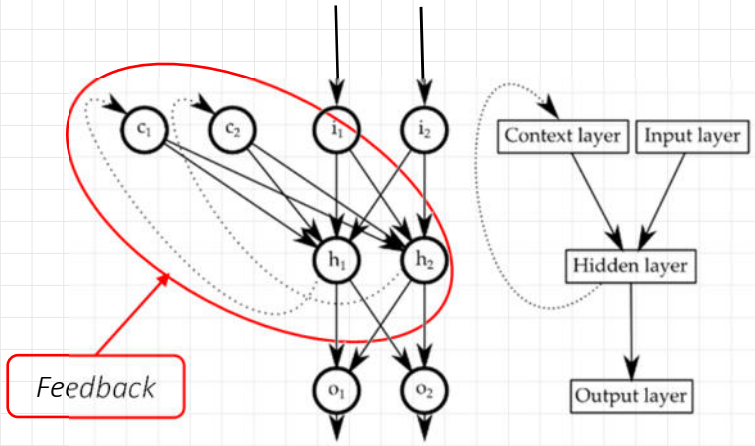
- The size of the test set should be about 10% to 30% of that of the training set.
- To avoid overfitting, the training set size should be at least 5 times the number of network weights.
- In theory, a single hidden layer is enough to approximate any continuous function, but in practice a further hidden layer often helps. More than 4 layers (an input, an output and two hidden) can work wonders but usually only in very controlled contexts.

Recurrent NN

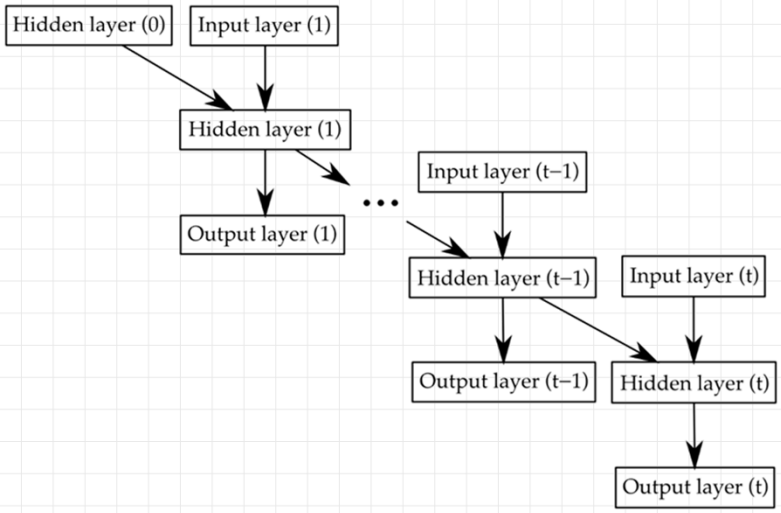
The most effective neural model for forecasting so far.

- They have at least one backward connection (*feedback loop*)
- They can keep activations even with no input → they have an *internal state*.
- Even with one single neuron, we can input a time series and get a series in the output.
- They can approximate any dynamical system.
- Very complex mathematical analysis.
- Very complex learning algorithms, hard to control. The most used one is Backpropagation Through Time (BTT).

RNN, example



RNN "unfolded"



RNN

RNNs compile preceding data in their context layer, they do not need an input neuron for each past value.

Their effect is not function approximation, but process modeling.

The prediction model is structured as:

$$output_{t+1} \approx \mathrm{RNN}(NetworkState, input_t, output_t)$$

where $NetworkState$ is represented by the activation of the neurons in the context layer.

RNN, activation functions

We can use any activation function we like in the recurrent neural network. Common choices are:

- Sigmoid function: $\frac{1}{1+e^{-x}}$
- Tanh function: $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
- Relu function: $\max(0, x)$

Learning, BPTT

BPTT(k) // pt is the target value at time step t:

1. while not (termination condition)
    1. Set all h to zero (hidden neurons states).
    2. Repeat for t = 0 to n-k
        1. Forward propagate the network over the unfolded network for k time steps to compute all h and y.
        2. Compute the error as: $e = y_{t+k} - p_{t+k}$
        3. Backpropagate the error across the unfolded network and update the weights.

Advantages and shortcomings of RNNs

RNNs have various advantages such as:

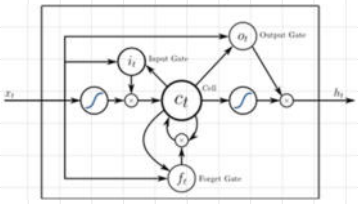
- Ability to handle sequence data.
- Ability to handle inputs of varying lengths.
- Ability to store or 'memorize' historical information.

The disadvantages are:

- The computation can be very slow.
- Vanishing gradient problem, where the gradients used to compute the weight update may get very close to zero preventing the network from learning new weights. The deeper the network, the more pronounced is this problem (common to deep MLP).

LSTM

Long Short-Term Memory (LSTM) networks are RNN trained by Backpropagation Through Time.



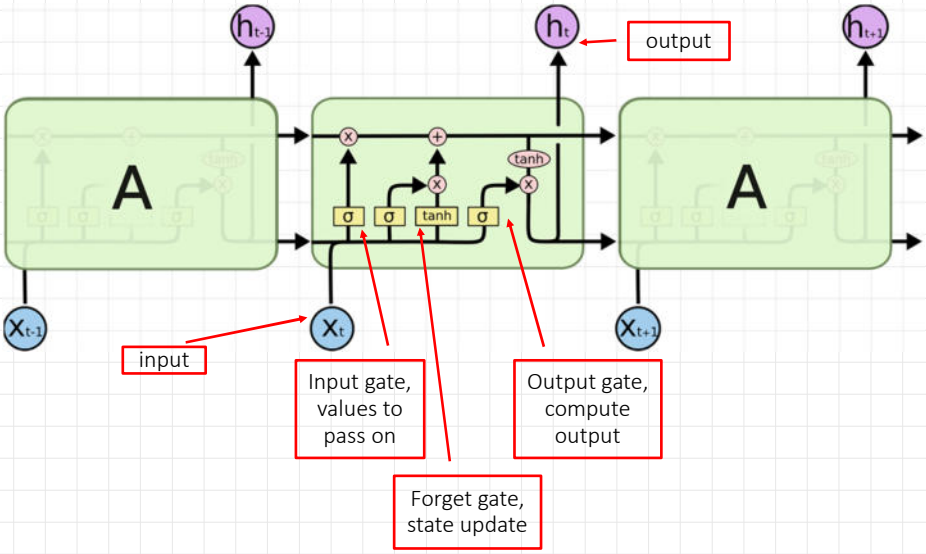
They do not have proper neurons, but memory blocks connecting layers.

A block is more complex than a normal neuron, it contains "gates" to maintain Internal state and output. Each gate works on the input sequence and uses an activation function that determines its contribution to internal state and output.

Three types of gates, whose weights must be defined during training:

- Forget Gate: decide which information is dropped.
- Input Gate: decide which input values contribute to the state.
- Output Gate: decide the output based on input and state.

LSTM



LSTM

An LSTM is a complicated, but single layer network. There are many implementation of LSTMs, for example:

$\mathbf{i}_t = \sigma(\mathbf{W}^{(x_i)}\mathbf{x}_t + \mathbf{W}^{(h_i)}\mathbf{h}_{t-1} + \mathbf{W}^{(c_i)}\mathbf{c}_{t-1} + \mathbf{b}(i))$

$\mathbf{f}_t = \sigma(\mathbf{W}^{(xf)}\mathbf{x}_t + \mathbf{W}^{(h_f)}\mathbf{h}_{t-1} + \mathbf{W}^{(c_f)}c_{t-1} + \mathbf{b}(f))$

$\mathbf{c}_t = \mathbf{f}_t \bullet \mathbf{c}_{t-1} + \mathbf{i}_t \bullet \tanh(\mathbf{W}^{(x_c)}\mathbf{x}_t + \mathbf{W}^{(h_c)}\mathbf{h}_{t-1} + \mathbf{b}^{(c)})$

$\mathbf{o}_t = \sigma(\mathbf{W}^{(x_o)}\mathbf{x}_t + \mathbf{W}^{(h_o)}\mathbf{h}_{t-1} + \mathbf{W}^{(c_o)}\mathbf{c}_t + \mathbf{b}^{(o)})$

$\mathbf{h}_t = \mathbf{o}_t \bullet \tanh(\mathbf{c}_t)$,

Where:

- σ is the logistic sigmoid function,
- $\mathbf{i}_t$, $\mathbf{f}_t$, $\mathbf{o}_t$ are vectors that represent ***input***, ***forget***, and ***output*** gates,
- $\mathbf{c}_t$ and $\mathbf{h}_t$ are vectors that represent "normal" and hidden state of the LSTM layer.

$\mathbf{W}$'s are weight matrices, where $\mathbf{W}^{(c_i)}$, $\mathbf{W}^{(c_f)}$, and $\mathbf{W}^{(c_o)}$ are diagonal (thus vectors, actually).

Gated Recurrent Units

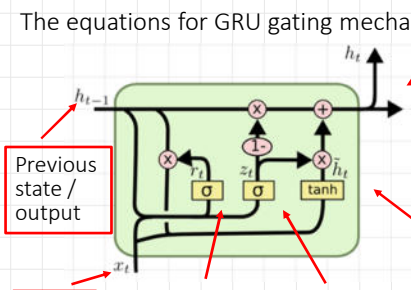
Gated Recurrent Units (GRUs) are another type of RNN, similar to LSTM but with fewer parameters: they do not have an output gate.

GRU gates are named:

- Update gate: decides the previous memory to keep.
- Reset gate: defines how to combine new input with previous memory.

Unlike LSTM in GRU, there is no persistent cell state distinct from the hidden state as in LSTM.

The equations for GRU gating mechanism:



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Fil Rouge, forecast errors

Train data 2004-2007 test data 2008.

Simple MLP model.

ACTUAL	MLP					
x_i	f_i	$x_i - f_i$	$\lvert x_i - f_i \rvert$	$(x_i - f_i)^2$		
752	529.55	222.45	222.45	49482.93	BIAS	38.37
468	374.37	93.63	93.63	8767.44	MAD	119.67
419	518.23	-99.23	99.23	9845.96	RMSE	134.27
725	788.36	-63.36	63.36	4014.96	MAPE	23.55

Easy to improve

LSTM example: jewelry stores

```
import pandas as pd, numpy as np, os
import matplotlib.pyplot as plt

os.chdir(os.path.dirname(os.path.abspath(__file__)))
df = pd.read_csv('gioiellerie.csv', header=0)
df["period"] = df["year"].map(str) +"-" + df["month"].map(str)
df['period'] = pd.to_datetime(df['period'], format="%Y-%m").dt.to_period('M')
# df = df.set_index('period')
aSales  = df['sales'].to_numpy()        # array of sales data
logdata = np.log(aSales)                # log transform
data    = pd.Series(logdata)            # convert to pandas series

plt.rcParams["figure.figsize"] = (10,8) # redefines figure size
plt.plot(data.values);plt.show()        # data plot

# train and test set
train_size  = int(len(data) -12)
test_size   = len(data) - train_size
train, test = data[0:train_size,:], data[train_size:len(data),:]

reconstruct = np.exp(np.r_[train,test]) # simple reconstruction (just for test)
```

LSTM example: jewelry stores

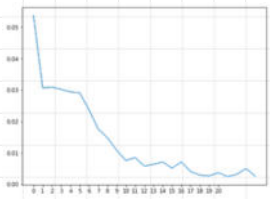
```
# ------------------------------------------------- neural LSTM forecast
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit_transform(train.values.reshape(-1, 1))
scaled_train_data = scaler.transform(train.values.reshape(-1, 1))
scaled_test_data  = scaler.transform(test.values.reshape(-1, 1))

from keras.preprocessing.sequence import TimeseriesGenerator
n_input = 12; n_features = 1
generator = TimeseriesGenerator(scaled_train_data, scaled_train_data, length=n_input,
batch_size=1)

from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM

lstm_model = Sequential()
lstm_model.add(LSTM(20, activation='relu', input_shape=(n_input, n_features), dropout=0.05))
lstm_model.add(Dense(1))
lstm_model.compile(optimizer='adam', loss='mse')
lstm_model.fit(generator,epochs=25)
lstm_model.summary()
```


LSTM example: jewelry stores



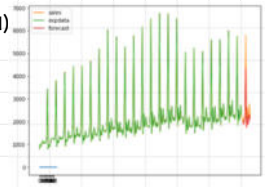
```
losses_lstm = lstm_model.history.history['loss']
plt.xticks(np.arange(0,21,1))   # convergence trace
plt.plot(range(len(losses_lstm)),losses_lstm); plt.show()

lstm_predictions_scaled = list()
batch = scaled_train_data[-n_input:]
curbatch = batch.reshape((1, n_input, n_features)) # 1 dim more
for i in range(len(test)):
 lstm_pred = lstm_model.predict(curbatch)[0]
 lstm_predictions_scaled.append(lstm_pred)
 curbatch = np.append(curbatch[:,1:,:],[[lstm_pred]],axis=1)

lstm_forecast = scaler.inverse_transform(lstm_predictions_scaled)
yfore = np.transpose(lstm_forecast).squeeze()

# recostruction
expdata = np.exp(train)    # unlog
expfore = np.exp(yfore)
plt.plot(df.sales, label="sales")
plt.plot(expdata,label='expdata')
plt.plot([None for x in expdata]+[x for x in expfore], label='forecast')
plt.legend()
plt.show()
```

Starts with the last train data and progressively slides with forecasts



LSTM example, torch, model

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from   sklearn.preprocessing import MinMaxScaler
import matplotlib,matplotlib.pyplot as plt

# Model
class LSTM(nn.Module):
   def __init__(self, input_size=1, hidden_size=24, num_layers=1):
      super().__init__()
      self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                          batch_first=True)
      self.fc = nn.Linear(hidden_size, 1)

   def forward(self, x):
      out, _ = self.lstm(x)
      return self.fc(out[:, -1, :])
```


LSTM example, torch, prelimin

```
if __name__ == "__main__":
    matplotlib.use("TkAgg")
    # Load dataset
    df = pd.read_csv("BoxJenkins.csv", usecols=[1])
    data = df.values.astype(float)

    # Normalize, needed
    scaler = MinMaxScaler()
    data_scaled = scaler.fit_transform(data)

    lag = 12
    X, y = create_dataset(data_scaled, lag)
    X_train = torch.tensor(X, dtype=torch.float32)
    y_train = torch.tensor(y, dtype=torch.float32)
```

LSTM example, torch, training

```
model     = LSTM()
loss_fn   = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# Training
for epoch in range(100):
   model.train()
   output = model(X_train)
   loss = loss_fn(output, y_train)
   optimizer.zero_grad()
   loss.backward()
   optimizer.step()
   if epoch % 10 == 0:
      print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```


LSTM example, torch, results

```
# Forecast next 12 values
model.eval()
fore = []
# unsqueeze adds a dimension, it is a pytorch unfortunate requirement
seq = torch.tensor(X[-1], dtype=torch.float32).unsqueeze(0)
for _ in range(12):
        with torch.no_grad():
            pred = model(seq)
            fore.append(pred.item())
            seq = torch.cat([seq[:, 1:], pred.unsqueeze(1)], dim=1)

# Inverse scale
forecast = scaler.inverse_transform(np.array(fore).reshape(-1, 1))

# Plot
plt.plot(df.values, label="Actual")
plt.plot(range(len(df), len(df) + 12), forecast, label="Forecast", color='red')
plt.legend()
plt.title("BoxJenkins LSTM series")
plt.show(block=True)  # block ensures it writes the window properly, it waits for it
print("fine")
```

Models, pros and cons

Model	Pros	Cons
Linear Regression	Ability to handle different time series components and features. High interpretability.	Sen… …ers. …s.
Exponential Smoothing	Ability to handle variabl… seasonality com… Automat…	…rvals.
ARIMA (Autoreg… Integrate… Moving Av…		…omate.
Dynamic Line… Model	…ers. …of the components.	Higher holdout error. Higher training and evaluation time.
Neural Network Model	…restrictions and assumptions. Ability to handle complex nonlinear patterns. High predictive power. Can be easily automated.	Low interpretability. Difficult to derive confidence intervals for the forecasts. Requires more data.

Comparison?
Check Diebold-Mariano
in statistics